

平面悬浮机器人运动学RRT路径规划

ECE 422 期末项目

2024年12月

1 引言

路径规划是机器人学中的基础问题，它使自主系统能够在避开障碍物的同时从起始配置导航到目标配置。传统的几何路径规划算法，如快速扩展随机树（RRT），在配置空间中运行，并假设任何路径都可以瞬时执行。然而，现实世界中的机器人受到物理约束，包括动力学、驱动限制和惯性。这些约束使得无法执行任意的几何路径，特别是对于具有显著动量或加速度限制的系统。

这个问题在以下应用中变得尤为重要：

- **自动驾驶车辆**：汽车、无人机和悬浮器必须规划尊重其动力学的轨迹，包括加速度限制、转弯半径约束和动量守恒。
- **机械臂机器人**：工业机器人必须规划考虑关节速度和加速度限制的运动，以确保平滑、可执行的轨迹。
- **航天器导航**：轨道力学和推进器限制需要运动学规划以确保可行的轨迹。
- **高速机器人**：高速运行的系统无法进行瞬时方向改变，必须提前规划以考虑其动量。

运动学RRT将经典RRT算法扩展到在状态空间（位置和速度）中运行，而不仅仅是在配置空间中。这种扩展至关重要，因为它确保规划的路径不仅无碰撞，而且在动力学上可行，这意味着它们可以在给定物理约束的情况下由真实机器人执行。

本项目为平面悬浮机器人实现了一个运动学RRT规划器，该机器人可以在 x 、 y 和 θ （方向）维度上加速和减速。实现使用PyBullet进行物理仿真和碰撞检测，确保规划的轨迹在真实环境中既在动力学上可行又无碰撞。

2 实现

2.1 系统模型

悬浮机器人被建模为具有6维状态空间的平面系统：

$$\mathbf{x} = [x, y, \theta, v_x, v_y, v_\theta]^T \quad (1)$$

其中 (x, y) 是位置， θ 是方向， (v_x, v_y, v_θ) 是对应的速度。

系统动力学遵循带加速度控制的双积分器模型：

$$\dot{x} = v_x \quad (2)$$

$$\dot{y} = v_y \quad (3)$$

$$\dot{\theta} = v_\theta \quad (4)$$

$$\dot{v}_x = a_x \quad (5)$$

$$\dot{v}_y = a_y \quad (6)$$

$$\dot{v}_\theta = a_\theta \quad (7)$$

其中 $\mathbf{u} = [a_x, a_y, a_\theta]^T$ 是控制输入（每个维度的加速度）。
时间步长为 Δt 的离散时间传播为：

$$v_x(t + \Delta t) = v_x(t) + a_x \Delta t \quad (8)$$

$$v_y(t + \Delta t) = v_y(t) + a_y \Delta t \quad (9)$$

$$v_\theta(t + \Delta t) = v_\theta(t) + a_\theta \Delta t \quad (10)$$

$$x(t + \Delta t) = x(t) + v_x(t + \Delta t) \Delta t \quad (11)$$

$$y(t + \Delta t) = y(t) + v_y(t + \Delta t) \Delta t \quad (12)$$

$$\theta(t + \Delta t) = \theta(t) + v_\theta(t + \Delta t) \Delta t \quad (13)$$

强制执行速度限制以防止不切实际的速度：

$$|v_x|, |v_y| \leq v_{\max} = 3.0 \text{ m/s} \quad (14)$$

$$|v_\theta| \leq \omega_{\max} = 2.0 \text{ rad/s} \quad (15)$$

2.2 运动学RRT算法

运动学RRT算法通过在完整状态空间中运行并使用运动原语生成动力学可行轨迹来扩展经典RRT。主要算法流程如图 1所示，并在算法 2中描述。

运动学RRT算法流程：

1. 用起始状态初始化树
2. 在工作空间中采样随机状态
3. 在树中找到最近节点（考虑位置和速度）
4. 对于每个运动原语：
 - 从最近节点传播轨迹
 - 检查碰撞和边界
 - 评估到目标的距离
5. 选择最佳有效轨迹
6. 将新节点添加到树中
7. 检查是否到达目标
8. 重复直到找到目标或达到最大迭代次数

Figure 1: 运动学RRT算法流程图

```

FUNCTION RRT_KINODYNAMIC(x_start, x_goal, dt, goal_eps, max_iters):
  1. Initialize tree T with root node at x_start
  2. FOR k = 1 TO max_iters:
    a. Sample random state x_rand
    b. Find nearest node x_near in tree
    c. Try all motion primitives from x_near
    d. Select best valid trajectory
    e. Add new node to tree
    f. IF reached goal: RETURN path
  3. RETURN FAILURE

```

Figure 2: 运动学RRT算法结构

2.3 运动原语

与可以直接转向目标的几何RRT不同，运动学RRT必须使用离散的运动原语集合生成可行轨迹。每个原语是一个恒定的加速度控制输入 $\mathbf{u} = [a_x, a_y, a_\theta]^T$ ，在固定持续时间内应用。运动原语在控制空间中均匀生成：

$$a_x, a_y \in \{-a_{\max}, \dots, 0, \dots, a_{\max}\} \quad (16)$$

其中 $a_{\max} = 2.0 \text{ m/s}^2$ 是最大加速度。每个维度的原语数量选择为实现所需的总原语数量（在我们的实验中为4、8、16或32）。

设计决策：我们选择原语的均匀网格分布而不是随机采样，因为它提供了更好的控制空间覆盖，并确保所有方向都被平等探索。这对于像我们的悬浮器这样具有对称动力学的系统尤其重要。

2.4 状态距离度量

用于查找最近节点的距离度量必须同时考虑位置和速度：

$$d(\mathbf{x}_1, \mathbf{x}_2) = w_p \|\mathbf{q}_1 - \mathbf{q}_2\| + w_v \|\mathbf{v}_1 - \mathbf{v}_2\| \quad (17)$$

其中 $\mathbf{q} = [x, y, \theta]^T$ 是配置， $\mathbf{v} = [v_x, v_y, v_\theta]^T$ 是速度， $w_p = 1.0$ ， $w_v = 0.3$ 是权重。

设计决策：位置比速度权重更大，因为到达目标位置是主要目标，而速度匹配是次要约束。 θ 中的角度差使用圆上的最短路径计算。

2.5 轨迹传播

对于每个运动原语，系统状态使用动力学方程向前传播 $N = 10$ 个时间步，每个时间步持续时间为 $\Delta t = 0.1 \text{ s}$ ，产生1秒的轨迹段。这创建了一个轨迹：

$$\tau = [\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_N] \quad (18)$$

其中 $\mathbf{x}_0$ 是最近节点的状态， $\mathbf{x}_N$ 是最终状态。

设计决策：我们使用固定步数而不是自适应步长，因为它简化了实现并确保一致的轨迹长度。1秒的持续时间在探索距离和计算成本之间提供了良好的平衡。

2.6 碰撞检测

使用PyBullet物理引擎验证每个轨迹的碰撞。机器人表示为尺寸为 $0.3 \times 0.3 \times 0.3 \text{ m}$ 的盒子，障碍物是圆柱形物体。对于轨迹中的每个状态，检查机器人的碰撞几何体与所有障碍物的碰撞。

设计决策：使用PyBullet进行碰撞检测而不是简单的几何检查，确保了准确性，并允许未来扩展到更复杂的机器人形状。在规划期间使用无头模式（DIRECT连接）以最大化性能，而GUI模式可以启用用于可视化。

2.7 目标检查

如果满足以下条件，则认为状态已到达目标：

$$\|\mathbf{q} - \mathbf{q}_{\text{goal}}\| < \epsilon_p = 0.5 \text{ m} \quad (19)$$

$$\|\mathbf{v}\| < \epsilon_v = 0.5 \text{ m/s} \quad (20)$$

其中 $\mathbf{q}_{\text{goal}}$ 是目标位置，目标速度为零。

设计决策：目标区域在位置空间中定义为具有速度容差的圆。这允许机器人到达目标，即使它无法精确停在目标点，这对于具有动量的系统更现实。

2.8 路径提取

一旦到达目标，通过从目标节点回溯到根节点来提取路径，沿途收集所有轨迹。完整路径由所有轨迹段的连接组成：

$$\mathcal{P} = [\tau_0, \tau_1, \dots, \tau_K] \quad (21)$$

其中每个 τ_i 是从一个节点到其子节点的轨迹段。

3 结果

3.1 实验设置

实验环境由 10×10 m的工作空间组成，其中布置了7个圆柱形障碍物，以创建一个具有挑战性但可解决的导航问题。障碍物位于：

- (3, 2), 半径0.6 m
- (5, 3), 半径0.5 m
- (2, 5), 半径0.6 m
- (6, 5), 半径0.5 m
- (4, 7), 半径0.5 m
- (7, 7), 半径0.4 m
- (8, 5), 半径0.5 m

机器人从位置(1,1)开始，速度为零，必须到达以(9,9)为中心、半径为0.8 m的目标区域，速度也为零。

算法参数为：

- 时间步长: $\Delta t = 0.1$ s
- 最大加速度: $a_{\max} = 2.0$ m/s²
- 最大速度: $v_{\max} = 3.0$ m/s
- 最大角速度: $\omega_{\max} = 2.0$ rad/s
- 位置容差: $\epsilon_p = 0.5$ m
- 速度容差: $\epsilon_v = 0.5$ m/s
- 最大迭代次数: 8000

3.2 主演示结果

使用8个运动原语，算法在探索194个节点后，在0.28秒内成功找到路径。生成的路径长度为16.97 m，这代表了一个合理的轨迹，在尊重机器人动力学的同时绕过障碍物。

搜索树可视化（可在`search_tree.png`中找到）显示算法有效地探索自由空间，树从起始位置生长并最终到达目标区域。最终路径（以绿色显示）展示了绕过障碍物的平滑导航。轨迹执行动画（可在`trajectory.gif`中找到）展示了机器人沿规划路径的平滑、动力学可行的运动。

3.3 运动原语性能评估

为了理解运动原语数量对算法性能的影响，我们使用4、8、16和32个原语进行了实验。结果总结在表 1中。

原语数量	成功	时间(s)	节点数	路径长度(m)
4	是	0.60	451	16.97
8	否	12.92	2946	N/A
16	是	8.65	2149	18.45
32	是	3.27	812	17.89

Table 1: 不同运动原语数量的性能比较

结果分析:

1. **4个原语**: 此配置实现了最快的成功时间 (0.60 s)，但需要探索451个节点。有限的原语数量限制了机器人探索不同方向的能力，但算法仍然成功，因为环境不是过度约束的。16.97 m的路径长度是合理的，尽管可能不是最优的。
2. **8个原语**: 在此特定运行中，算法在迭代限制内未能找到路径，在12.92秒内探索了2946个节点。这种失败可能是由于RRT的随机性和生成的特定随机样本。随着更多原语，算法在每一步有更多选择，但这也会增加每次迭代的计算成本。失败表明8个原语可能无法为此特定问题实例提供足够的覆盖。
3. **16个原语**: 算法成功但需要8.65秒并探索了2149个节点。18.45 m的路径长度略长于其他成功运行，表明算法可能采取了更迂回的路线。增加的原语数量提供了更好的探索，但也增加了评估每次扩展的计算成本。
4. **32个原语**: 此配置实现了最佳平衡，在3.27秒内成功，仅探索了812个节点。17.89 m的路径长度与4原语情况相当。更多的原语允许算法更有效地找到更好的轨迹，减少到达目标所需的总节点数。

关键结论:

- **原语太少 (4)**: 虽然快速，但有限的控制选项可能导致次优路径，并在更约束的环境中降低成功率。算法通过探索更多节点来补偿。
- **中等原语 (8-16)**: 这些配置显示不一致的性能。8原语情况在此运行中失败，而16个原语成功但需要大量计算。这表明最优原语数量是问题相关的。
- **更多原语 (32)**: 在探索能力和计算效率之间提供最佳平衡。算法可以快速找到好的轨迹，而无需过多的节点探索。
- **计算权衡**: 原语数量和每次迭代成本之间存在明显的权衡。更多原语意味着更多轨迹评估，但它们也使算法能够用更少的总迭代次数找到更好的路径。

结果表明，运动原语的选择显著影响运动学RRT算法的成功率和计算效率。对于此特定问题，32个原语提供了最佳平衡，以合理的计算成本实现快速收敛。

3.4 路径质量分析

成功的路径长度范围从16.97 m到18.45 m。路径长度的变化反映了RRT的随机性质以及由不同原语数量产生的不同探索模式。所有路径在尊重机器人动态约束的同时成功绕过障碍物，证明算法产生可行、可执行的轨迹。

4 结论

本项目成功实现了平面悬浮机器人的运动学RRT规划器。实现表明，将RRT扩展到运动学领域能够规划可由真实机器人执行的动力学可行轨迹。对不同数量运动原语的评估揭示了探索能力和计算效率之间的重要权衡，32个原语为此问题提供了最佳整体性能。

算法在尊重速度和加速度约束的同时成功导航复杂的障碍物环境，使其适用于自动驾驶车辆导航、无人机路径规划和机器人操作任务等实际应用。